Javaはどのように動くのか～図解でわかるJVMの仕組み

達人が教える
Webパフォーマンスチューニング
ISUCONから学ぶ高速化の実践

第5回

チューニングのために理解しておきたいGCの4つのアルゴリズム

伊藤智博（いとうちひろ）　2013-03-27　35　シェア

記事の内容　3分
なぜアルゴリズムを学ぶのか
GCを2つのフェーズに分けて実行する ～マーク＆スイープGC
バラバラに配置されていたオブジェクトを連続して配置する ～コンパクション
GCと同時にコンパクションを実行する ～コピーGC
「一世代ヒープ」の問題を考える
「世代別ヒープ」でGCの時間を減らす

なぜアルゴリズムを学ぶのか

GCによる停止時間が長くなり、アプリケーションの処理時間が短くなると、業務に使える時間が短くなってしまいます。その問題を解決するために、GCをチューニングすることで、アプリケーションの停止時間を短くすることが考えられます。

その際大事なのは、GCのアルゴルズムを把握しておくことです。

GCのチューニングを行うときは、GCで行われている処理の内、どの処理に時間がかかっているかをモニタリング⇒分析⇒チューニングする、という流れになります。しかし、GCのアルゴリズムを知らないと、モニタリング結果を見てもどこに問題があるかがわからず、分析やチューニングを行うことができません。

今回は、以下の4つのアルゴリズムをご紹介します。

- マーク＆スイープGC
- コンパクション
- コピーGC
- 世代別GC

GCのアルゴリズムはJVMの実装によって異なりますが、多くの場合、上記4つのアルゴリズムが組み合わされて実装されています。

GCを2つのフェーズに分けて実行する ～マーク＆スイープGC

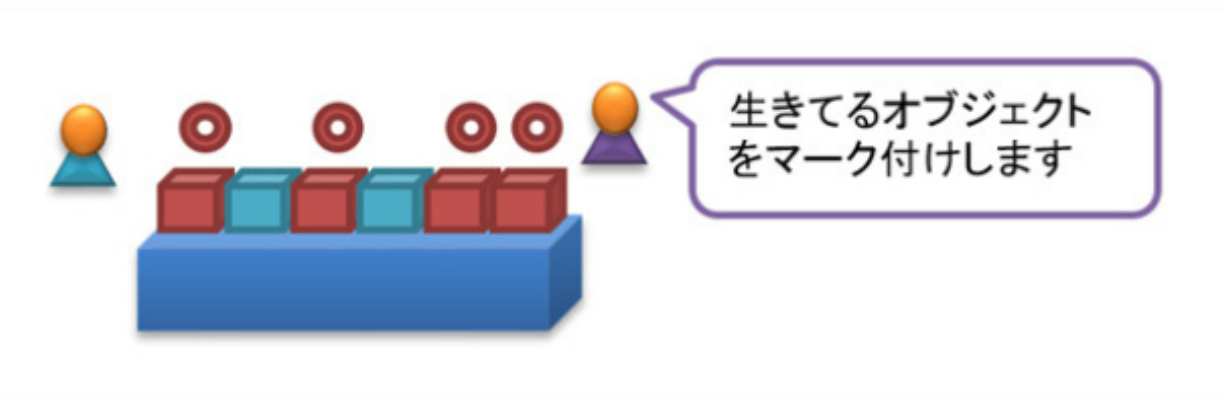
これまでの連載からイメージしやすいGCアルゴリズムは、マーク＆スイープGCでしょう。マーク＆スイープGCでは、GCを以下の2つのフェーズに分けて実行します。

1つ目のフェーズ
⇒ゴミとなっていないオブジェクトを探す（マークフェーズ）

2つ目のフェーズ
⇒ゴミとなったオブジェクトを削除するフェーズ（スイープフェーズ）

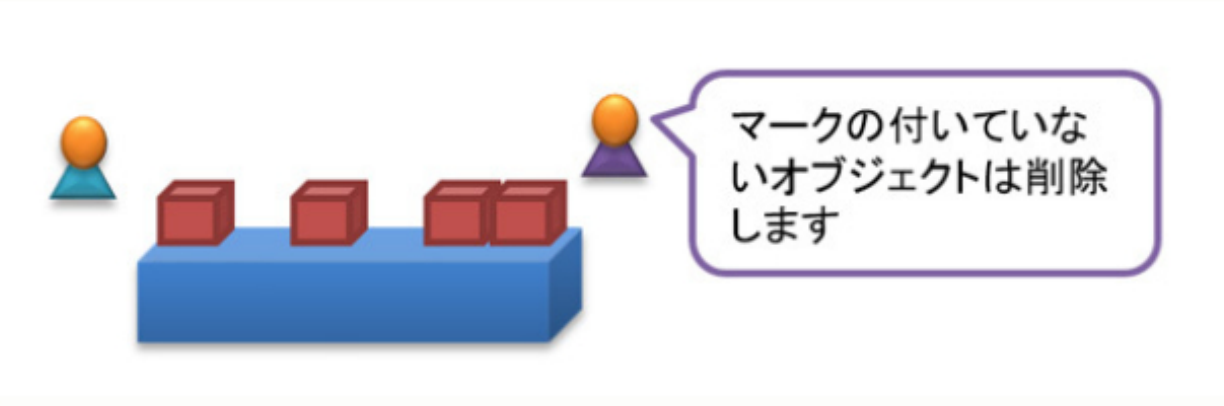
マークフェーズでは、ヒープ内にあるオブジェクトの参照をたどっていき、生きてるオブジェクトをマークしていきます。JVMは、マークフェーズが終わったときにマークされていないものを死んだオブジェクトとして判定します。

図1　マークフェーズ後


生きてるオブジェクトをマーク付けします

スイープフェーズでは、マークが付いていないオブジェクトを回収することで、ヒープ領域を開放します。スイープフェーズが終わると、死んだオブジェクトに割り当てられていた領域が、新たなオブジェクトに割り当てられるようになります。

図2　スイープフェーズ後

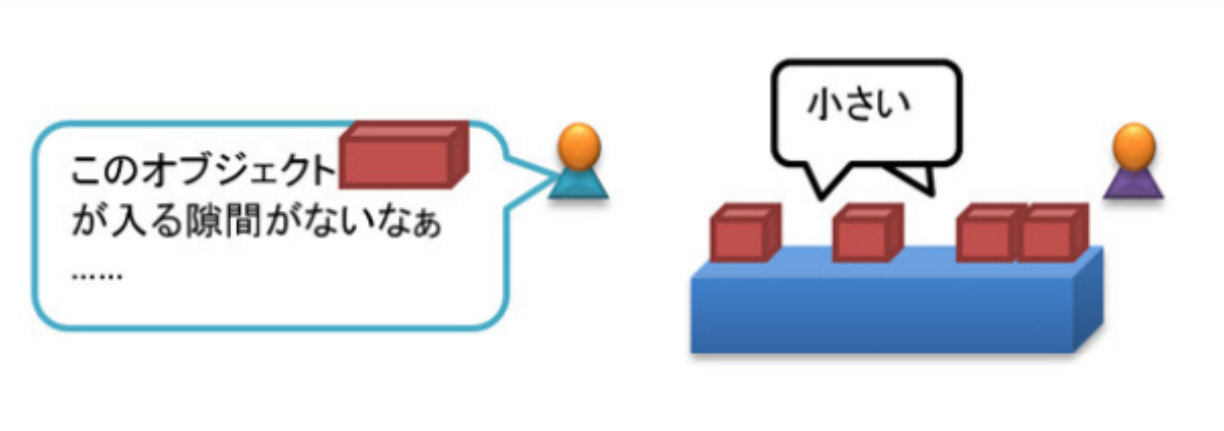

マークの付いていないオブジェクトは削除します

マーク＆スイープGCでは、死んだオブジェクトをスイープした領域がそのまま空き領域になります。そのため、ヒープの中に空き領域が点在してしまいます。この空き領域は、新たなオブジェクトを割り当てる際に、空いている領域のサイズまでのオブジェクトしか生成できません。そのように、空き容量が点在することを「断片化」と言います。

断片化が多発してしまうと、どのような問題があるのでしょうか？

まず、ヒープの空き容量があったとしても、オブジェクトが生成できる場所が限定されてしまい、このオブジェクトが入る空き領域を探すのに時間がかかります。さらに最悪のケースでは、オブジェクトを生成できる場所が見つからなくなると、アプリケーションが予期せず停止してしまうかもしれません。

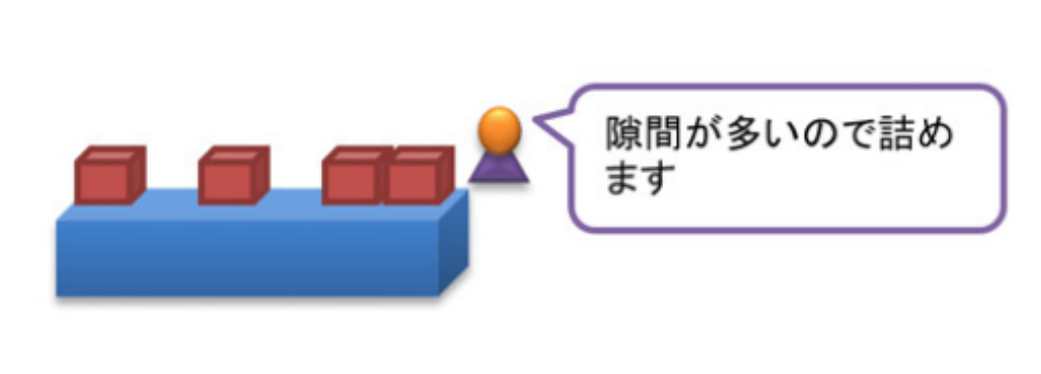
図3　断片化によって大きなオブジェクトが生成できない


このオブジェクトが入る隙間がないなぁ……
小さい

バラバラに配置されていたオブジェクトを連続して配置する ～コンパクション

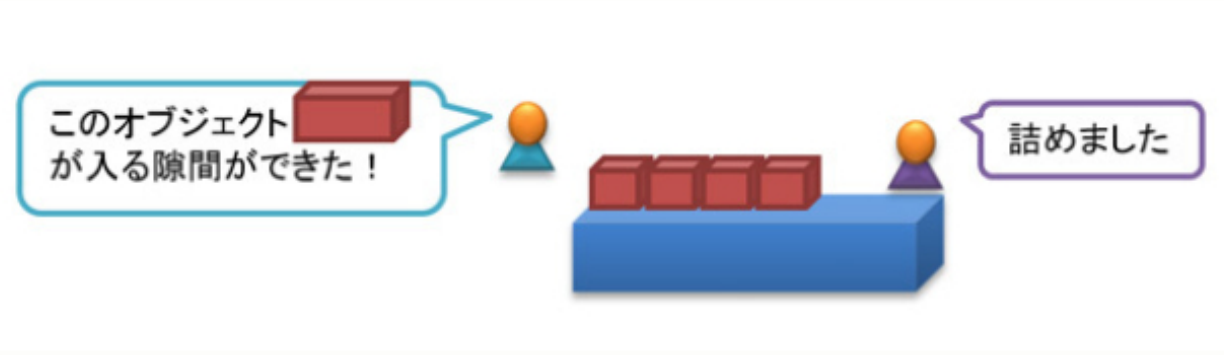
ヒープ内にバラバラに配置されていたオブジェクトを、ヒープの一端に連続して配置することが考えれます。この動作を「コンパクション」と呼びます。

図4　コンパクション前


隙間が多いので詰めます

コンパクションにより、空き領域をつなげられ、オブジェクトを生成する場所をかんたんに見つけられます。

図5　コンパクション後


このオブジェクトが入る隙間ができた！
詰めました

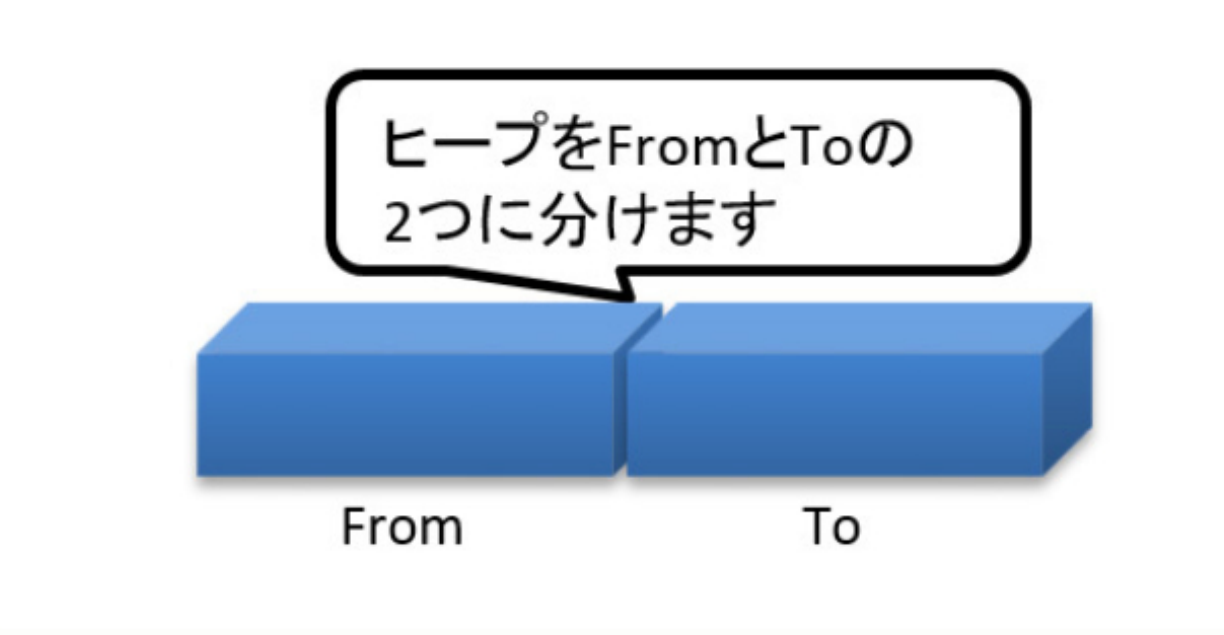
GCと同時にコンパクションを実行する ～コピーGC

断片化の対策としてコンパクションを解説しましたが、そもそもヒープが断片化しないGCアルゴリズムはあるのでしょうか？

その答えが「コピーGC」です。コピーGCでは、GCと同時にコンパクションと同じ処理を行っているため、ヒープが断片化することはありません。

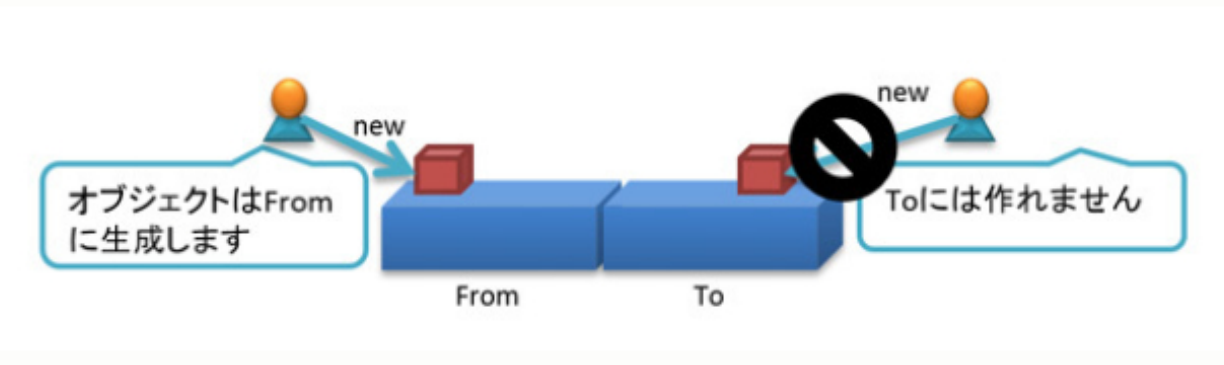
マーク＆スイープGCでは、ヒープを1部屋として使用していました。一方、コピーGCでは、ヒープをFrom領域とTo領域の2つに分けて使用します。

図6　ヒープを分割


ヒープをFromとToの2つに分けます
From
To

アプリケーションが新たなオブジェクトを生成すると、“必ず”From領域に新たなオブジェクトを割り当てます。アプリケーションは“絶対に”To領域を使えません。

図7　オブジェクトの生成

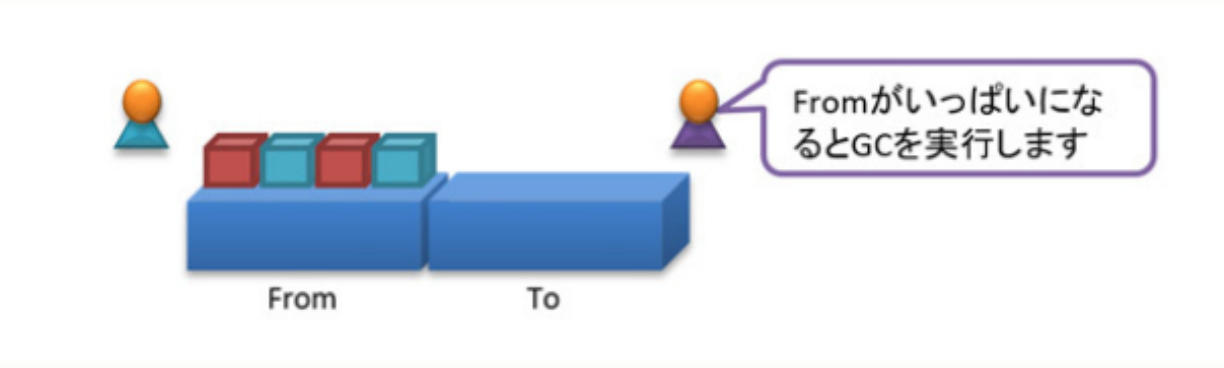

new
オブジェクトはFromに生成します
From
To
new
Toには作れません

それでは、To領域はどのような場合に使われるのでしょうか？

答えは「コピーGCの実行中」です。

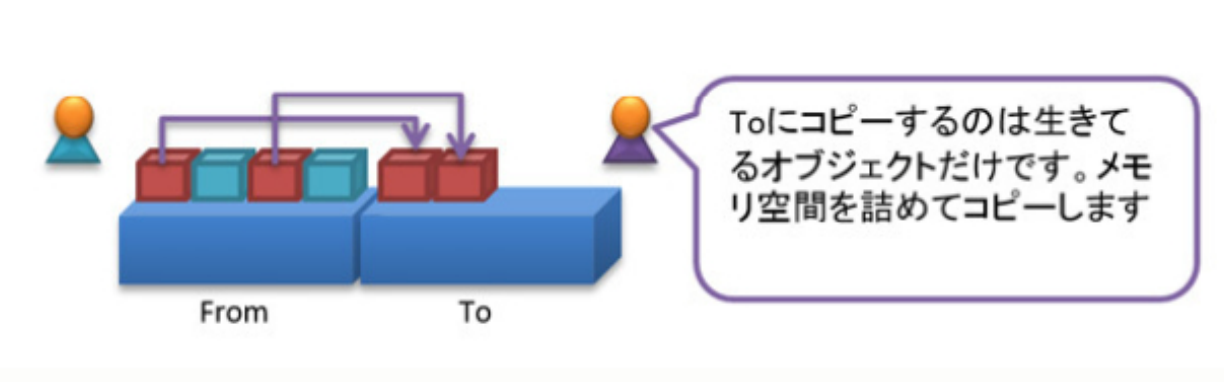
アプリケーションが実行されると、From領域の残り空き容量が少なくなっていきます。そうすると、JVMはコピーGCを行います。

図8　コピーGCの開始


From
To
FromがいっぱいになるとGCを実行します

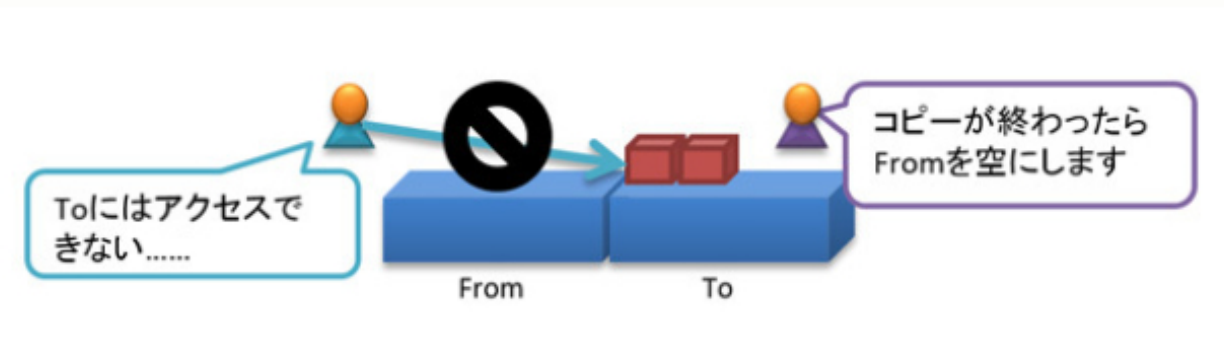
コピーGCは、From領域にあるオブジェクトのうち、生きているオブジェクトを探します。生きてるオブジェクトが見つかると、そのオブジェクトをTo領域の端から順にコピーしていきます。

図9　オブジェクトのコピー


From
To
Toにコピーするのは生きてるオブジェクトだけです。メモリ空間を詰めてコピーします

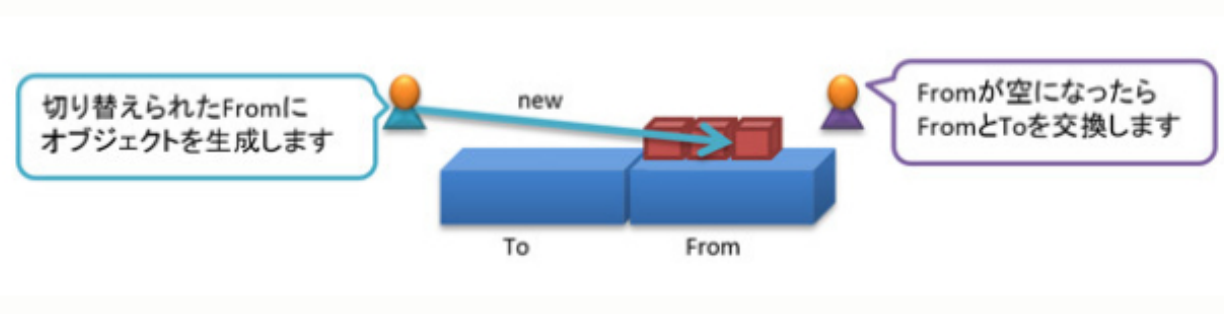
From領域をすべて探し終わると、すべての生きているオブジェクトはTo領域にコピーされ、From領域は空になります。しかし、アプリケーションはTo領域を使うことができないため、To領域にコピーされた生きているオブジェクトにアクセスすることができません。そのため、このままではアプリケーションが正常に動作することができなくなってしまいます。

図10　オブジェクトのコピー後


Toにはアクセスできない……
From
To
コピーが終わったらFromを空にします

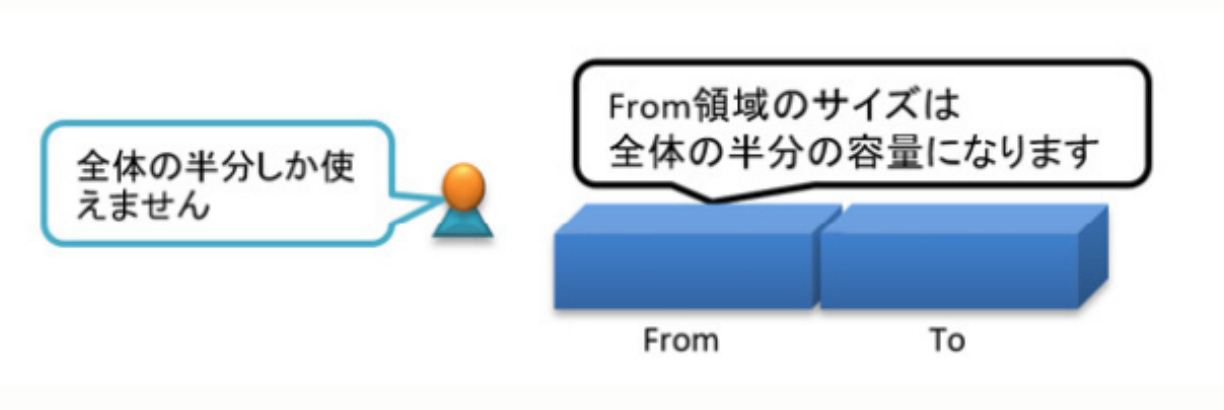
この問題を解消するために、コピーGCでは、領域の名前を変更する動作が行われます。具体的には、これまでTo領域であった領域がFrom領域になり、これまでFrom領域だった領域がTo領域となります。アプリケーションは、これまでTo領域だった新しいFrom領域にアクセスすることで、コピーされた生きてるオブジェクトにアクセスすることも、オブジェクトを生成することもできるようになります。

図11　領域の切り替え


切り替えられたFromにオブジェクトを生成します
new
To
From
Fromが空になったらFromとToを交換します

コピーGCでは、GC時にTo領域の前から順に使用されていきます。これにより、前述したように、GCと同時にコンパクションと同じ処理を行われ、ヒープが断片化することはありません。そのようなメリットがある一方、コピーGCには弱点もあります。アプリケーションはFrom領域しか使用できないため、使用できる容量が半分となってしまうのです。

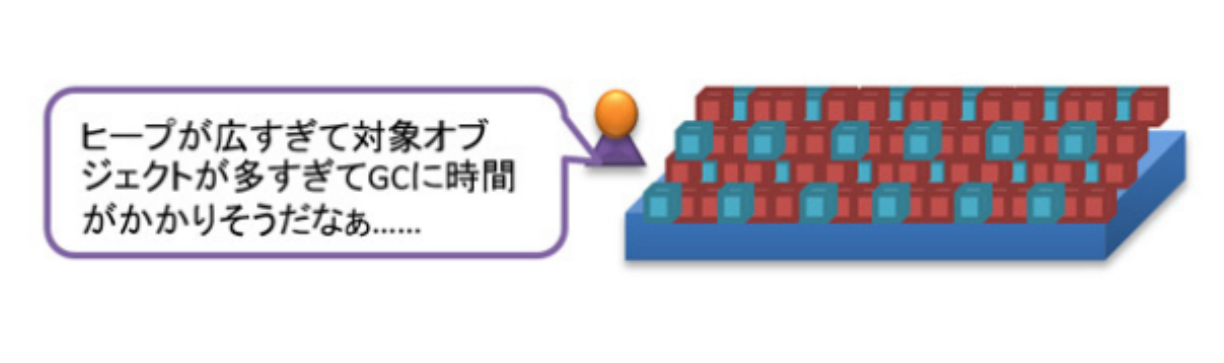
図12　コピーGCで使用できる容量


全体の半分しか使えません
From領域のサイズは全体の半分の容量になります
From
To

「一世代ヒープ」の問題を考える

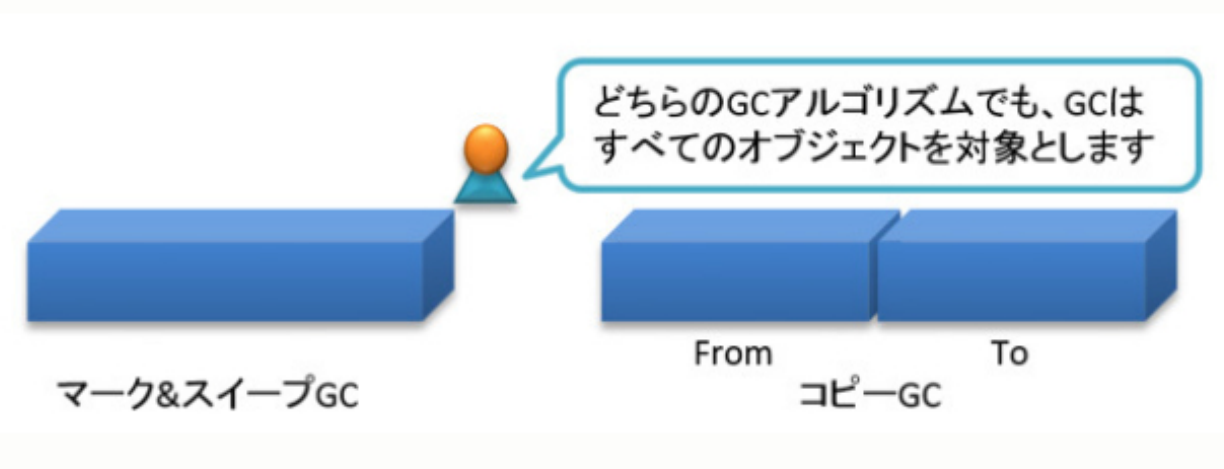
これまでのように、すべてのオブジェクトをGC対象とすることを「一世代ヒープ」と言います。一世代ヒープでは、起動時に生成されたオブジェクトも、GC間際に生成されたオブジェクトも同じヒープに格納されています。

図13　一世代ヒープ


ヒープが広すぎて対象オブジェクトが多すぎてGCに時間がかかりそうだなぁ……

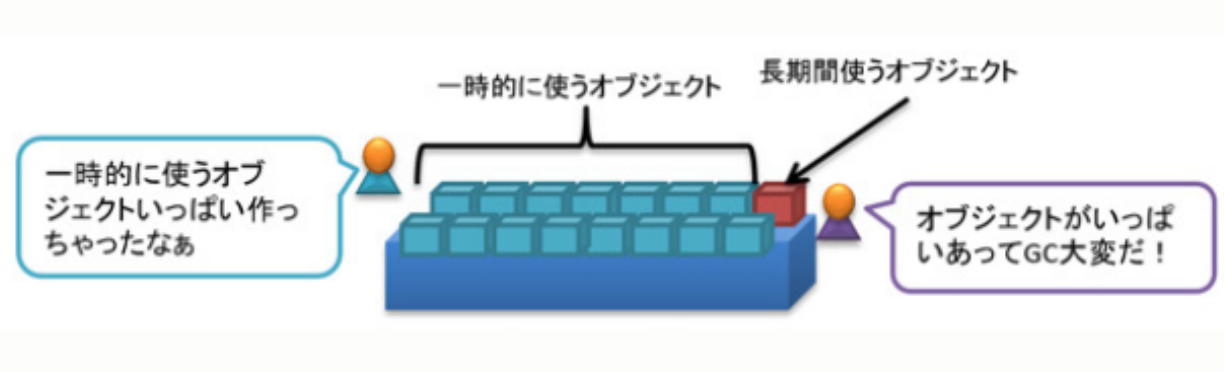
アプリケーションでは、多くのオブジェクトはわずかな時間だけ利用され、長時間利用されるオブジェクトはごく少数です。

図14　一般的なオブジェクトの割合


どちらのGCアルゴリズムでも、GCはすべてのオブジェクトを対象とします
マーク&スイープGC
From
To
コピーGC

一世代GCでは、ヒープサイズが大きくなるとヒープに格納されるオブジェクト数が増え、GCの時間が長くなり、アプリケーションの停止時間が長くなってしまいます。

図15　ヒープが広すぎる


一時的に使うオブジェクト
長期間使うオブジェクト
一時的に使うオブジェクトいっぱい作っちゃったなぁ
オブジェクトがいっぱいあってGC大変だ！

「世代別ヒープ」でGCの時間を減らす

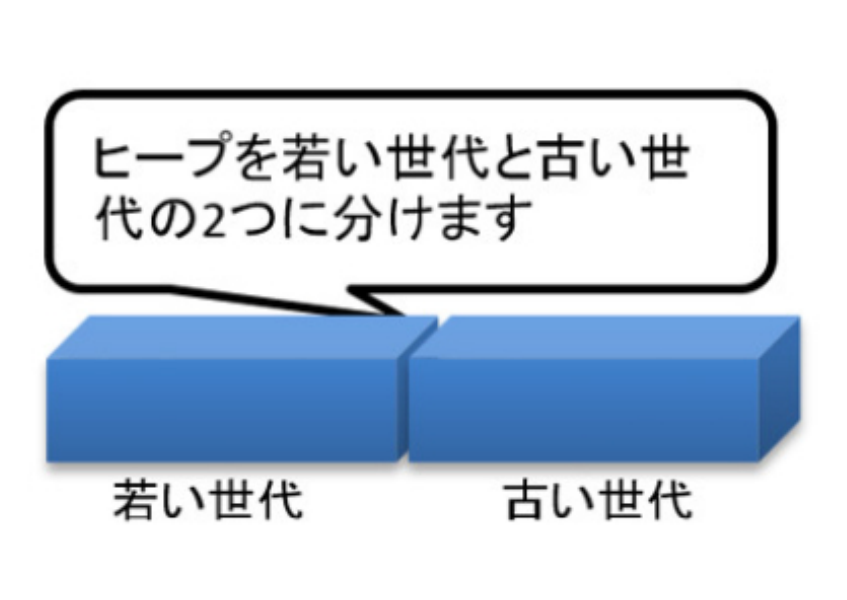
では、ヒープサイズを大きくしても停止時間をできるかぎり短くするには、どうしたらいいのでしょうか？

答えは、わずかな時間だけ利用されるオブジェクトと長時間利用されるオブジェクトを分けて管理・GCし、GC対象となるオブジェクトを限定すれば、GCの時間を減らすことです。

そのような考え方を取り入れたのが「世代別ヒープ」です。

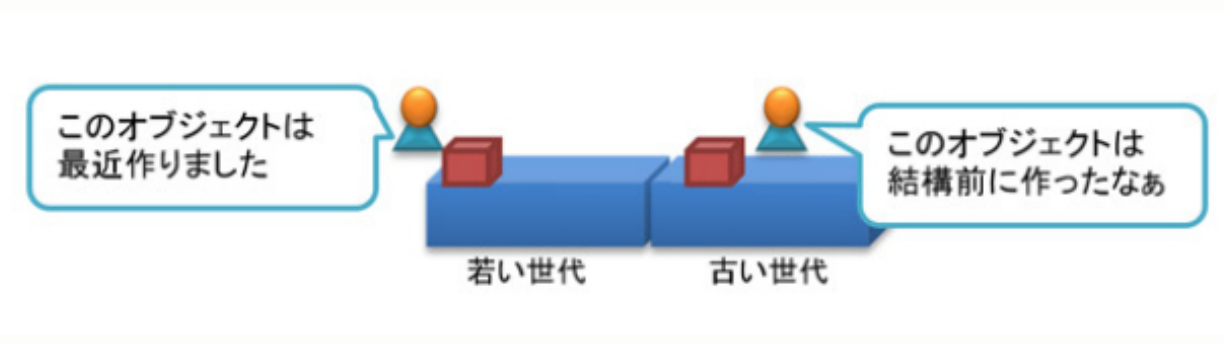
世代別ヒープでは、オブジェクトがGCされた回数を年齢（age）と呼びます。また、若い領域をYoung領域、古い領域をOld領域と表現することもあります。

図16　世代別ヒープ


ヒープを若い世代と古い世代の2つに分けます
若い世代
古い世代

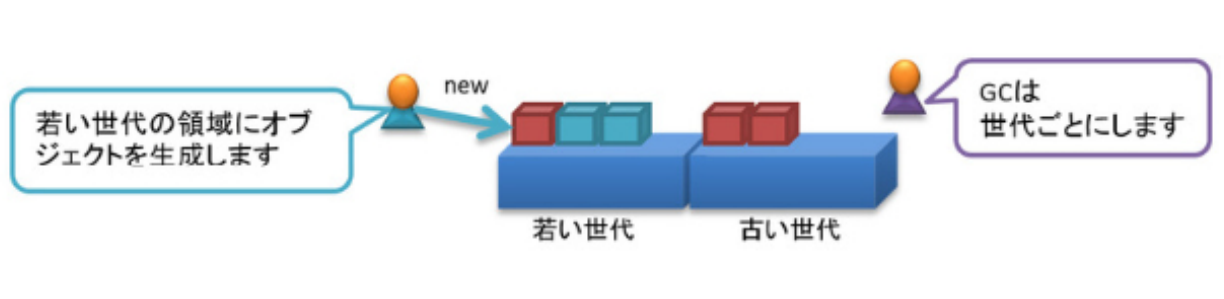
若い領域には、オブジェクトが生成されてから間もないオブジェクトが格納されます。古い領域には、オブジェクトが生成されてから、一定回数GCが行われたオブジェクトが格納されます。

図17　各領域とオブジェクトの関係


このオブジェクトは最近作りました
若い世代
古い世代
このオブジェクトは結構前に作ったなぁ

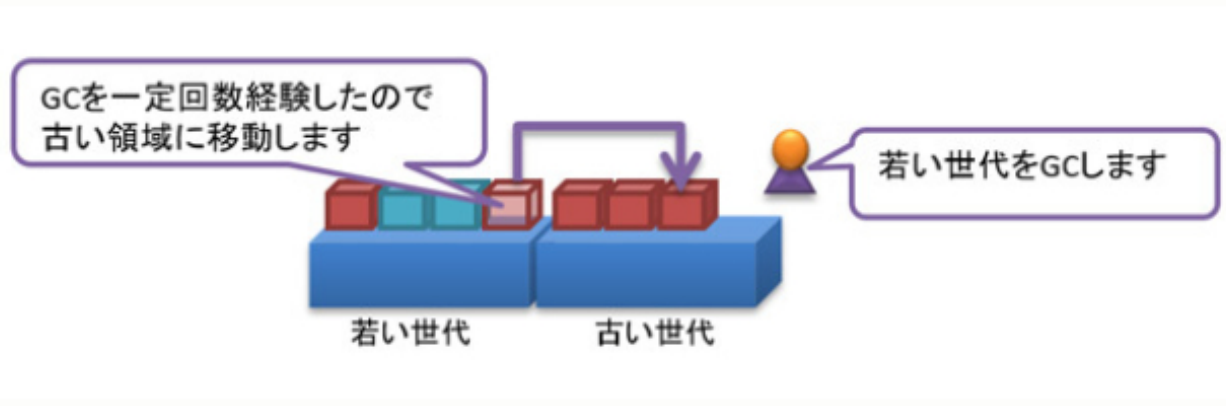
アプリケーションがオブジェクトを生成すると、若い領域のヒープを割り当てます（※）。

図18　世代別ヒープへのオブジェクトの生成


若い世代の領域にオブジェクトを生成します
new
若い世代
古い世代
GCは世代ごとにします

若い領域がいっぱいになると、若い世代のGCが行われます。若い領域にある死んだオブジェクトはここで解放されますが、若い領域にある生きたオブジェクトがある一定回数のGCを経験すると、若い領域から古い領域へコピーされます。

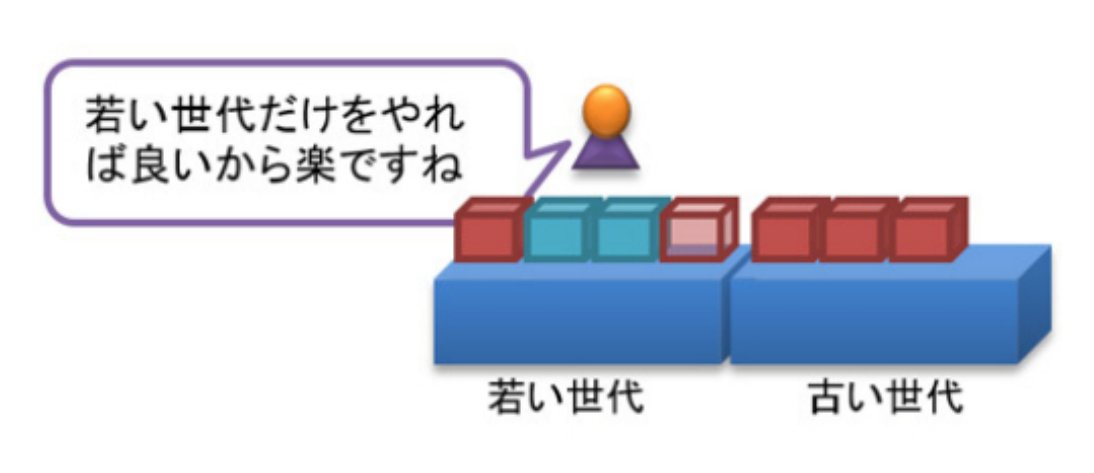
図19　若い世代のGC


GCを一定回数経験したので古い領域に移動します
若い世代
古い世代
若い世代をGCします

若い領域のヒープがいっぱいになると、若い領域だけをGCすることで、対象となるヒープやオブジェクトが少なくなります。

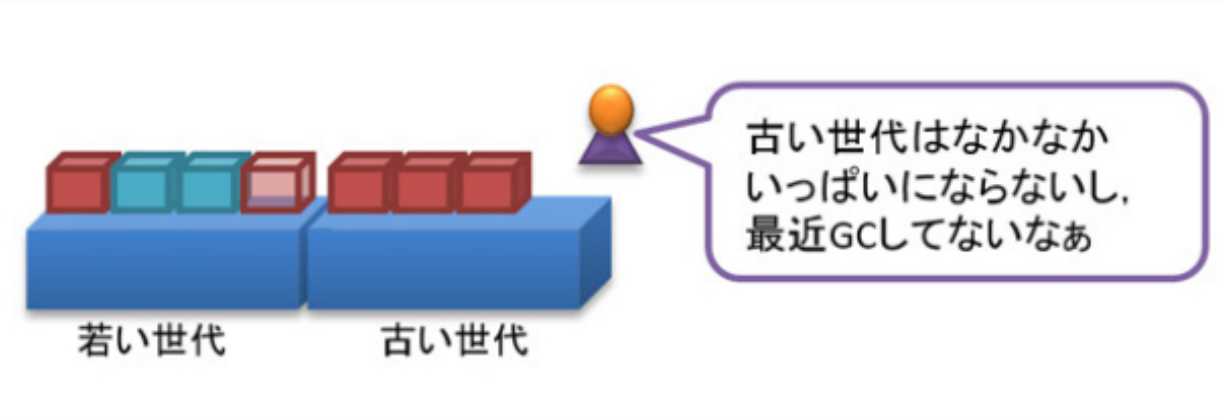
こうすることで、一世代で管理する場合に比べてGC時間が短くなり、アプリケーションの停止時間も短くなるのです。

図20　若い世代だけを対象とするGC


若い世代だけをやれば良いから楽ですね
若い世代
古い世代

若い領域のGCで一時的に使用されるオブジェクトを解放することで、古い領域に移動するオブジェクトは少なくなり、古い領域のGCが行われる回数を削減することができます。

図21　古い世代はめったにGCされない


若い世代
古い世代
古い世代はなかなかいっぱいにならないし、最近GCしてないなぁ

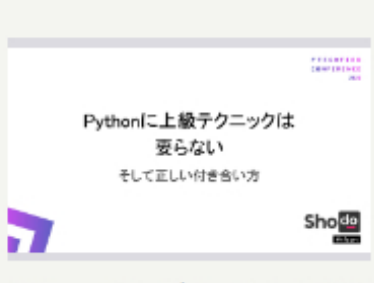
今回で、HotspotやJRockitなど、JVMの実装に問われないGCの基礎知識は終わりになります。次回は、Hotspot JVMではこれらの実装がどのようになっているかをご紹介します。

> ※）
> ただし、JVMの実装によっては、大きすぎるサイズのオブジェクトが古い領域にいきなり割り当てられることもありますのでご注意ください。

プログラミング一般

おすすめ記事

Vue Fes Japan Online 2022
Vue.jsの日本最大級カンファレンス「Vue Fes Japan Online 2022」、10/16にYouTube Live上で開催
2022-10-05

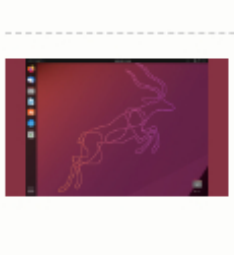

Pythonに上級テクニックは要らない
Pythonに上級テクニックは要らない（そして正しい付き合い方）
～　TechFeed　Conference 2022講演より
2022-10-05

LINE
Technology & Engineering
LINEの各種プロダクト・サービスを生み出すエンジニアたちに取材し、開発の秘訣に迫ります！
広告・宣伝

ニャンゴロウ先生が図解で教える
AWSのキホン
最新のAWS情報を知りたいなら、まずこの連載で！
広告・宣伝

記事・ニュース一覧

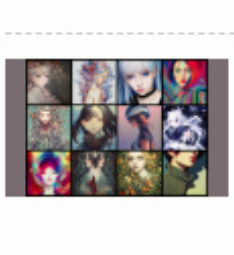
年々進化するチートシートと万全の準備体制でついに栄冠を獲得！ISUCON12優勝チームNaruseJun特別インタビュー
馮富久，LINE株式会社
2022-10-19


第735回
Ubuntu 22.10の変更点
あわしろいくや
2022-10-19

第2回
偽陽性と偽陰性
～自動テストの信頼性をむしばむ現象を理解する～
和田卓人
2022-10-19

プログラムを作った！それからどうする？プログラミングの技術を学ぶ以外にどういった知識を持っていれば良いのかを学びましょう。
増井敏克
広告・宣伝

第74回
正規表現の脆弱性「ReDoS」徹底解説
～原理と対策から、Perlでの最適化まで（2）
藤浪大弥，牧大輔［監修］，福本貴之［監修］，松木雅幸［監修］，大沢和宏［監修］
2022-10-18


第174回
DALL·E2、Midjourney、Stable Diffusion……「画像生成AI」が変えるもの、変えないもの
Lançamento
2022-10-18

第182回
知っておくとちょっと便利なMySQLのfind_in_set関数
木村浩一郎
2022-10-18

Are you ready to skate yet?
―Debianリリースチーム、Debian 12 "Bookworm"の最初のフリーズを2023年1月12日に予定
階戸アキラ
2022-10-17

HTML＆CSSのスキルアップを目指せ！コード写経で「作る喜び」を体験しよう！
服部雄樹
広告・宣伝

第74回
正規表現の脆弱性「ReDoS」徹底解説
～原理と対策から、Perlでの最適化まで（1）
藤浪大弥，牧大輔［監修］，福本貴之［監修］，松木雅幸［監修］，大沢和宏［監修］
2022-10-17

入門Kueue
～KubernetesのBatchワークロード最前線～
岩井佑樹
2022-10-17

11/16～18オンライン開催「FlutterKaigi 2022」参加受付開始、タイムテーブル

も公開

2022-10-14

LINEとヤフー、11/17、18にオンライン技術カンファレンス「Tech-Verse」を共同開催

2022-10-14

VirtualBox 7.0がリリース、仮想マシンのフル暗号化をサポート

階戸アキラ

2022-10-14

電子書籍サイトの案内 技術評論社刊行の電子書籍を読もう！PDFやEPUB形式で提供しており、様々な環境に対応しています。

広告・宣伝

MarkdownエディタObsidian v1.0.0がリリース

2022-10-14

Ubuntu 22.10のRelease Candidateに向けた作業

吉田史

2022-10-14

Alan Wu氏「YJITはRubyプロセス実行から終了まで全体のパフォーマンス向上を目指す」

～RubyKaigi 2022 3日目キーノート

花川直己

2022-10-13

やっぱり写真はこれ。Google Pixel 7シリーズ正式発表

傍島康雄

2022-10-13

まつもとゆきひろさん「価値を生み続けるための鍵はコミュニティ ―Contribute to Ruby―」

～RubyKaigi 2022 2日目キーノート

田中健太郎

2022-10-12

第734回

UbuntuでSBOM（ソフトウェア部品表）を作る方法

柴田充也

2022-10-12

『数学』が苦手な人でも大丈夫！ ― 『図形と数の並びで学ぶプログラミング基礎』

竹中要一

広告・宣伝

---

［2］Ruby biz Grand prix 2021で受賞した企業の声

編集部

2022-10-12

最近のWeb3Dひとめぐり～TechFeed Conference 2022講演より

emadurandal［スピーカー］，豊田陽介［執筆］

2022-10-12

技術評論社

Copyright © 2007-2022 All Rights Reserved by Gijutsu-Hyoron Co., Ltd.

ページ内容の全部あるいは一部を無断で利用することを禁止します。個別にライセンスが設定されている記事等はそのライセンスに従います。